

Interprocess Communication

Interprocess Communication

- Consider shell pipeline
 - `cat chapter1 chapter2 chapter3 | grep tree`
 - 2 processes
 - Information sharing
 - Order of execution

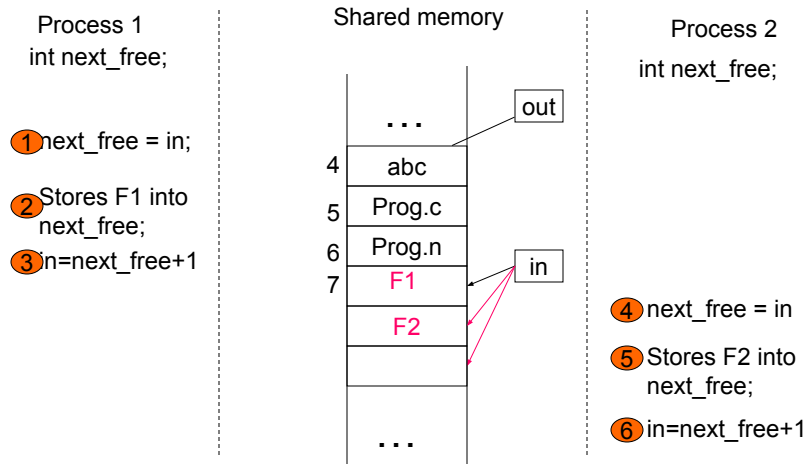
Interprocess Communication

IPC issues

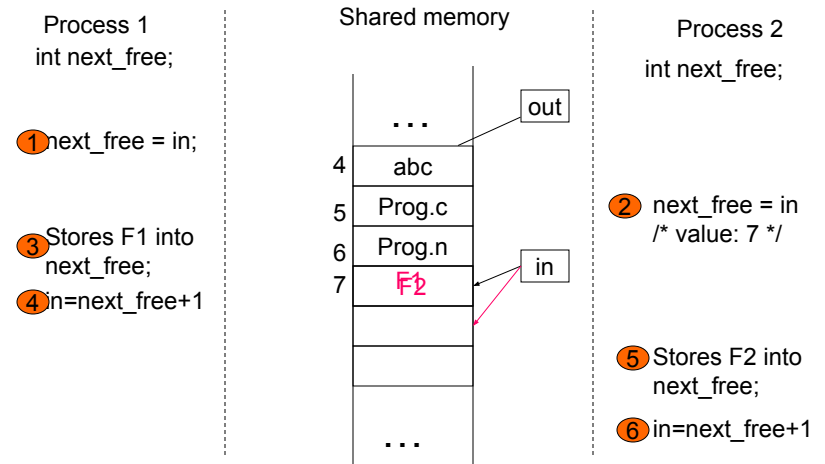
- Processes within a system may be **independent** or **cooperating**
- Cooperating process can affect or be affected by other processes
- Reasons for cooperating processes:
 - Information sharing
 - Computation speedup
 - Modularity
 - Convenience
- Cooperating processes require a **mechanism** to exchange data and information

1. How one process **passes** information to another ?
2. How to make sure that two or more processes do not get into each other's way when engaging in **critical** activities?
3. How to do proper **sequencing** when **dependencies** are present?
 - Ans 1: easy for threads, for processes different approaches (e.g., message passing, shared memory)
 - Ans 2 and Ans 3: same problems and same solutions apply for threads and processes
 - **Mutual exclusion & Synchronization**

Spooling Example: Correct



Spooling Example: Races



Better Coding?

- In previous code


```
for(;;){
    int next_free = in;
    slot[next_free] = file;
    in = next_free+1;
}
```
- What if we use one line of code?


```
for(;;){
    slot[in++] = file
}
```

When Can process Be switched?

- After each **machine** instruction!
- `in++` is a C/C++ statement, translated into **three** machine instructions:
 - load mem, R
 - inc R
 - store R, mem
- Interrupt (and hence process switching) can happen in between.

Race condition

- Two or more processes are reading or writing some **shared** data and the final result depends on who runs precisely when
- Very hard to Debug

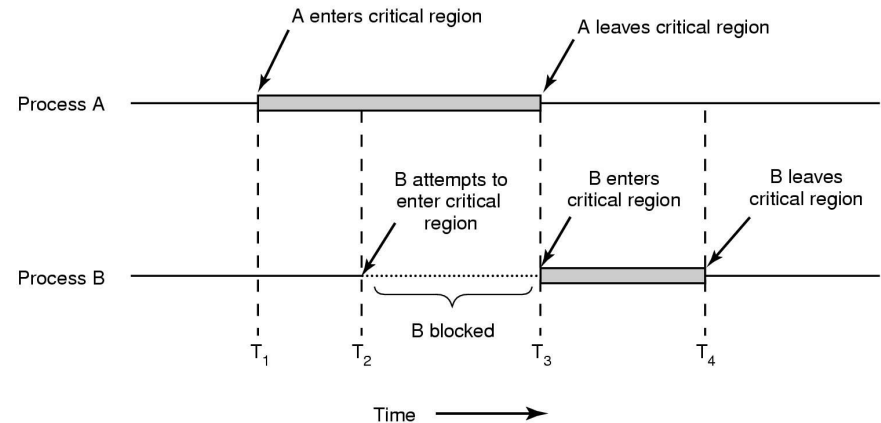
Critical Region

- That **part** of the program that do critical things such as accessing shared memory
- Can lead to race condition

Solution Requirement

- 1) No two processes **simultaneously** in **critical region**
- 2) No assumptions made about speeds or numbers of CPUs
- 3) No process running **outside** its critical region may **block** another process
- 4) No process must wait forever to enter its critical region

Solution Requirement



Mutual exclusion With Busy Waiting

- Possible Solutions
 - Disabling Interrupts
 - Lock Variables
 - Strict Alternation
 - Peterson's solution
 - TSL

Lock Variables

```
int lock = 0;
while (lock);
lock = 1;
//EnterCriticalSection;
    access shared variable;
//LeaveCriticalSection;
lock = 0;
```

Does the above code work?

Disabling Interrupts

- How does it work?
 - Disable all interrupts just after entering a critical section
 - Re-enable them just before leaving it.
- Why does it work?
 - With interrupts disabled, no clock interrupts can occur
 - No switching can occur
- Problems:
 - What if the process forgets to enable the interrupts?
 - Multiprocessor? (disabling interrupts only affects one CPU)

```
while (true) {
    /* disable interrupts */;
    /* critical section */;
    /* enable interrupts */;
    /* remainder */;
}
```

Lock Variables

```
int lock = 0;
while (lock);
lock = 1;
//EnterCriticalSection;
    access shared variable;
//LeaveCriticalSection;
lock = 0;
```

Still doesn't work!

Strict Alternation

Problem

```

while (TRUE) {
    while (turn != 0)    /* loop */ ;
    critical_region();
    turn = 1;
    noncritical_region();
}

while (TRUE) {
    while (turn != 1)    /* loop */ ;
    critical_region();
    turn = 0;
    noncritical_region();
}
    
```

(a) Process 0

(b) Process 1

Proposed solution to critical region problem

- Busy waiting: Continuously testing a variable until some value appear
 - Wastes CPU time
- Violates condition 3
 - When one process is much slower than the other

Peterson's solution

- Consists of 2 procedures
- Each process has to call
 - `enter_region` with its own process # before entering its C.R.
 - And `leave_region` after leaving C.R.

```

do {
    enter_region(process#)
    critical section
    leave_region(process#)
    remainder section
} while (TRUE);
    
```

Peterson's solution (for 2 processes)

```

#define FALSE 0
#define TRUE 1
#define N      2                /* number of processes */

int turn;                        /* whose turn is it? */
int interested[N];              /* all values initially 0 (FALSE) */

void enter_region(int process); /* process is 0 or 1 */
{
    int other;                  /* number of the other process */

    other = 1 - process;        /* the opposite of process */
    interested[process] = TRUE; /* show that you are interested */
    turn = process;             /* set flag */
    while (turn == process && interested[other] == TRUE) /* null statement */ ;
}

void leave_region(int process) /* process: who is leaving */
{
    interested[process] = FALSE; /* indicate departure from critical region */
}
    
```

Peterson's Solution: Analysis(1)

- Let Process 1 is not interested and Process 0 calls enter_region with 0
- So, turn = 0 and interested[0] = true and Process 0 is in CR
- Now if Process 1 calls enter_region, it will hang there until interested[0] is false. Which only happens when Process 0 calls leave_region i.e. leaves the C.R.

Peterson's Solution: Analysis(2)

- Let both processes call enter_region **simultaneously**
- Say turn = 1. (i.e. Process 1 stores **last**)
- Process 0 enters critical region: while (turn == 0 && ...) returns **false** since turn = 1.
- Process 1 loops until process 0 exits: while (turn == 1 && interested[0] == true) returns true.
- It works fine!!

22

Busy Waiting: Problems

- **Waste** CPU time since it sits on a tight loop
- May have **unexpected** effects:
 - **Priority Inversion Problem**

Example:

- 2 **Cooperating** Processes: H (high priority) and L (low priority)
- *Scheduling rule*: H is run whenever it is ready
- Let L in C. R. and H is ready and wants to enter C.R.
- Since H is ready it is given the CPU and it starts busy waiting
- L will never gets the chance to leave its C.R.
- H loops forever

Sleep & wakeup

- When a process has to **wait**, change its state to **BLOCKED**
- Switched to **READY** state, when it is OK to retry entering the critical section
- Sleep is a **system call** that causes the caller to block
 - be suspended until another process wakes it up
- Wakeup system call has one parameter, the process to be awakened.
- Let's illustrate the use of sleep & wakeup with an example: **The producer consumer problem**

Sleep and Wakeup

Producer Consumer Problem

- Also called bounded-buffer problem
- Two (or $m+n$) processes share a **common** buffer
- One (or m) of them is (are) **producer**(s): put(s) information in the buffer
- One (or n) of them is (are) **consumer**(s): take(s) information out of the buffer
- Trouble and solution
 - Producer wants to put but buffer **full**- Go to **sleep** and **wake up** when consumer takes one or more
 - Consumer wants to take but buffer **empty**- go to sleep and wake up when producer puts one or more

```
#define N 100
int count = 0;

/* number of slots in the buffer */
/* number of items in the buffer */

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        if (count == N) sleep();
        insert_item(item);
        count = count + 1;
        if (count == 1) wakeup(consumer);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        if (count == 0) sleep();
        item = remove_item();
        count = count - 1;
        if (count == N - 1) wakeup(producer);
        consume_item(item);
    }
}
```

Sleep and Wakeup

```
#define N 100
int count = 0;

/* number of slots in the buffer */
/* number of items in the buffer */

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        if (count == N) sleep();
        insert_item(item);
        count = count + 1;
        if (count == 1) wakeup(consumer);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        if (count == 0) sleep();
        item = remove_item();
        count = count - 1;
        if (count == N - 1) wakeup(producer);
        consume_item(item);
    }
}
```

Producer-consumer problem

Sleep and Wakeup: Race condition

- **Busy waiting problem is resolved but the following race condition exists**
- **Unconstrained access to *count***
 - CPU is given to P just after C has **read** count to be 0 but not yet gone to sleep.
 - P calls wakeup
 - Result is **lost** wake-up signal
 - Both will sleep forever

Semaphores



- A new variable type
- A kind of **generalized** lock
 - First defined by Dijkstra in late 60s
 - Main synchronization **primitive** used in original UNIX
- Semaphores are like integers, except
 - No negative values
 - Only operations allowed are *up* and *down* – can't read or write value, except to set it initially

Semaphores: Operation

- Operation “down”:
 - if value > 0; value-- and then continue.
 - if value = 0; process is put to sleep without completing the down for the moment
 - Checking the value, changing it, and possibly going to sleep, is all done as an **atomic action**.
- Operation “up”:
 - increments the value of the semaphore addressed.
 - If one or more process were sleeping on that semaphore, one of them is chosen by the system (e.g. at **random**) and is allowed to complete its *down*
 - The operation of incrementing the semaphore and waking up one process is also **indivisible**
 - No process ever blocks doing an *up*.

Semaphores: Types

- **Counting semaphore.**
 - The value can range over an unrestricted domain
- **Binary semaphore**
 - The value can range only between 0 and 1.
 - On some systems, binary semaphores are known as **mutex** locks as they provide mutual exclusion

Semaphores: Atomicity

- Operations must be **atomic**
 - Two *down*'s together can't decrement value below zero
 - Similarly, process going to sleep in *down* won't miss wakeup from *up* – even if they both happen at same time

Producer & consumer

```
#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

/* number of slots in the buffer */
/* semaphores are a special kind of int */
/* controls access to critical region */
/* counts empty buffer slots */
/* counts full buffer slots */

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        down(&empty);
        down(&mutex);
        insert_item(item);
        up(&mutex);
        up(&full);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item();
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}
```